

Pontifícia Universidade Católica de Campinas

Escola Politécnica

Engenharia de Software

Padrões e Arquitetura de Software

**EFC1: Refatoração Guiada por SOLID,
Clean Code e Padrões GoF**

24005297 – Caio dos Santos Cunha

24020425 – Fillipe de Souza Faria

24012527 – Hugo Silva Pinheiro

24008474 – Leticia Akemi Sumida

24004942 – Lucas de Campos Ranzani

24011554 – Luis Gustavo Oliveira da Silva

(<https://github.com/caiodscunha/Trabalho-ECF1/tree/develop>)

Prof. Dr. Douglas H. S. Abreu

Campinas — 2026

1	Identificação das Violações SOLID.....	4
	SRP — Single Responsibility Principle.....	4
	Violação 1: Classe Sis acumula mais de 6 responsabilidades (legacy.py, linhas 6–160)	4
	Violação 2: proc_pag acopla autorização de pagamento e mudança de status (linhas 116–139)	4
	OCP — Open/Closed Principle	5
	Violação 1: Pagamentos por if/elif hard-coded (linhas 116–139)	5
	Violação 2: Tipos de desconto por if/elif hard-coded (linhas 18–26)	5
	LSP — Liskov Substitution Principle.....	5
	Violação 1: PedEspecial.add_ped suprime o desconto do cliente VIP (linhas 163–181).....	5
	Violação 2: PedEspecial.upd_st ignora a máquina de estados do pedido (linhas 183–190) ...	6
	ISP — Interface Segregation Principle.....	6
	Violação 1: Interface monolítica de Sis força dependência de SQLite em todos os consumidores (linhas 6–160)	6
	Violação 2: gerar_rel acopla geração de conteúdo e escrita em arquivo (linhas 90–114)	7
	DIP — Dependency Inversion Principle.....	7
	Violação 1: Sis cria sqlite3.connect() diretamente (linhas 8–13)	7
	Violação 2: Notificações implementadas como print() no fluxo de negócio (linhas 40–47)	7
2	Soluções Implementadas.....	8
	SRP — Extração em serviços especializados	8
	OCP — Strategy para métodos de pagamento	8
	OCP — Strategy para descontos de itens e clientes	9
	LSP — Eliminação da herança problemática de PedEspecial	9
	ISP — Criação de 8 interfaces focadas	10
	DIP — Injeção de dependência explícita em todos os serviços.....	10
3	Padrões GoF Aplicados.....	11
	3.1 Strategy.....	11
	3.2 Repository.....	11
	3.3 Observer (Pub-Sub).....	12
	3.4 Factory Method	13
	3.5 Decorator	13
4	Diagrama UML de Classes.....	14

5 Melhorias de Clean Code	16
Nomenclatura.....	16
Extração de métodos	16
Eliminação de duplicação.....	16
Abstrações criadas.....	16
6 Extensões Implementadas.....	17
Extensão 1 — Pagamento em criptomoeda (taxa de 2%)	17
Extensão 2 — Notificação WhatsApp para todos os tipos de cliente.....	17
Extensão 3 — Desconto progressivo por volume (3+ unidades = 15% off).....	18
7 Saída dos Testes	19
Resultado da execução	19
Cobertura de código (coverage.py)	19
8 Relatório de Métricas	20
Lint — Ruff.....	20
Tipagem estática — mypy --strict.....	20
Complexidade ciclomática — Radon CC.....	20
Resumo das métricas.....	20
9 Reflexão Metacognitiva	21
O princípio que mais gerou dúvida	21
Como usamos IA neste trabalho.....	21
Bibliografia.....	22

REPOSITÓRIO DO PROJETO: <https://github.com/caiodscunha/Trabalho-ECF1/tree/develop>

1 Identificação das Violações SOLID

O código legado consiste em uma única classe `Sis` (`legacy.py`) com 223 linhas e uma subclasse `PedEspecial` que viola Liskov. A seguir são identificadas ao menos duas violações por princípio.

SRP — Single Responsibility Principle

Violação 1: Classe `Sis` acumula mais de 6 responsabilidades (`legacy.py`, linhas 6–160)

A classe `Sis` gerencia simultaneamente: (1) conexão e schema SQLite, (2) cálculo de totais e descontos, (3) persistência de pedidos, (4) processamento de pagamento, (5) envio de notificações por canal e tipo de cliente, (6) validação de estoque e (7) geração de relatórios. Uma classe com tantas responsabilidades muda por múltiplos motivos não relacionados.

```
# legacy.py - linhas 6-13: conexão + schema na mesma classe que processa pagamento
class Sis:
    def __init__(self):
        self.db = sqlite3.connect('loja.db')
        self.c = self.db.cursor()
        self.c.execute('CREATE TABLE IF NOT EXISTS ped (...)')
        self.db.commit()
```

Impacto: qualquer mudança de banco de dados (ex: migrar para PostgreSQL) exige modificar a mesma classe que contém regras de desconto e lógica de notificação.

Violação 2: `proc_pag` acopla autorização de pagamento e mudança de status (linhas 116–139)

O método `proc_pag` autoriza o pagamento e imediatamente chama `upd_st` para alterar o estado do pedido dentro do mesmo fluxo. Autorizar um pagamento e alterar o estado do pedido são razões de mudança independentes. Regras de negócio financeiro e máquina de estados do pedido não deveriam coexistir no mesmo método.

```
# legacy.py - linhas 116-132
def proc_pag(self, id, m, vl):
    if m == 'cartao':
        print('Processando pagamento com cartao...')
        self.upd_st(id, 'aprovado') # efeito colateral: muda estado
        return True
    elif m == 'pix':
        print('Gerando QR Code PIX...')
        self.upd_st(id, 'aprovado') # efeito colateral: muda estado
        return True
```

Impacto: reprocessar um pagamento falho afeta o estado do pedido como efeito colateral. Testes de pagamento precisam verificar mudanças de status.

OCP — Open/Closed Principle

Violação 1: Pagamentos por if/elif hard-coded (linhas 116–139)

Adicionar criptomoeda ou qualquer novo método de pagamento exige editar `proc_pag` com mais um `elif`, violando OCP: a classe precisa ser aberta para modificação ao invés de apenas para extensão.

```
# legacy.py — linhas 123-139
if m == 'cartao':
    print('Processando pagamento com cartao...')
    self.upd_st(id, 'aprovado')
    return True
elif m == 'pix':
    print('Gerando QR Code PIX...')
    self.upd_st(id, 'aprovado')
    return True
elif m == 'boleto':
    print('Gerando boleto...')
    return True
else:
    print('Metodo de pagamento invalido!')
    return False
```

Impacto: cada nova forma de pagamento exige testes de regressão em toda a cadeia `if/elif`.

Violação 2: Tipos de desconto por if/elif hard-coded (linhas 18–26)

O cálculo de subtotal por tipo de item usa uma cadeia `if/elif` fixada no código. Qualquer novo tipo de item (ex: `'desc30'`, `'cashback'`) obriga edição direta do método.

```
# legacy.py — linhas 18-26
for i in its:
    if i['tipo'] == 'normal':
        tot += i['p'] * i['q']
    elif i['tipo'] == 'desc10':
        tot += i['p'] * i['q'] * 0.9
    elif i['tipo'] == 'desc20':
        tot += i['p'] * i['q'] * 0.8
    elif i['tipo'] == 'frete_gratis':
        tot += i['p'] * i['q']
```

LSP — Liskov Substitution Principle

Violação 1: `PedEspecial.add_ped` suprime o desconto do cliente VIP (linhas 163–181)

`Sis.add_ped` aplica o desconto de 5% para clientes VIP ao final do cálculo. `PedEspecial` sobrescreve `add_ped` com uma lógica que aplica taxa de 15% mas não aplica o desconto VIP, a pós-condição herdada é enfraquecida. Substituir `Sis` por `PedEspecial` faz um cliente VIP perder o desconto a que tem direito.

```
# legacy.py — linhas 163-181
class PedEspecial(Sis):
    def add_ped(self, n, its, t):
```

```

tot = 0
for i in its:
    if i['tipo'] == 'normal':
        tot += i['p'] * i['q']
    ...
    tot = tot * 1.15      # taxa especial
# NÃO aplica: if t == 'vip': tot *= 0.95
self.c.execute('INSERT INTO ped ...')
self.db.commit()

```

Impacto: código polimórfico produz totais diferentes para os mesmos dados de entrada. Hierarquia de herança frágil.

Violação 2: PedEspecial.upd_st ignora a máquina de estados do pedido (linhas 183–190)

Sis.upd_st envia notificações (email, SMS, pontos de fidelidade) ao transitar entre estados. PedEspecial sobrescreve upd_st executando apenas o UPDATE no banco, sem emitir nenhum evento. A invariante do pai, transições geram eventos de negócio, é quebrada pela subclasse.

```

# legacy.py – linhas 183-190
class PedEspecial(Sis):
    def upd_st(self, id, s):
        p = self.get_ped(id)
        if p:
            self.c.execute('UPDATE ped SET st=? WHERE id=?', (s, id))
            self.db.commit()
            print(f'Pedido especial {id} -> {s}') # sem email, sem SMS,
sem pontos

```

Impacto: clientes VIP perdem pontos de fidelidade; auditoria de notificações fica inconsistente.

ISP — Interface Segregation Principle

Violação 1: Interface monolítica de Sis força dependência de SQLite em todos os consumidores (linhas 6–160)

Não há segregação entre as interfaces de cálculo, persistência, pagamento e relatório. Qualquer código que precise apenas de get_ped() recebe também proc_pag(), gerar_rel() e a conexão SQLite, pois tudo está na mesma classe. Não existe ponto de corte entre o que cada colaborador realmente precisa.

```

# Para calcular total, consumidor recebe também:
def get_ped(self, id): ... # → banco de dados
def proc_pag(self, ...): ... # → pagamento
def gerar_rel(self, ...): ... # → arquivo em disco

```

Impacto: testes de cálculo dependem de SQLite real; mock de toda a classe é necessário para qualquer teste isolado.

Violação 2: gerar_rel acopla geração de conteúdo e escrita em arquivo (linhas 90–114)

O método gerar_rel gera o conteúdo do relatório e o persiste em disco na mesma operação, sem permitir que o chamador decida o destino da saída. Geração de conteúdo e persistência são responsabilidades distintas que deveriam estar em interfaces separadas.

```
# legacy.py – linhas 90-114
def gerar_rel(self, tipo):
    with open('rel_vendas.txt', 'w') as f:
        f.write(...) # grava arquivo obrigatoriamente
```

Impacto: impossível obter o relatório como string; envio por e-mail exige copiar o arquivo gerado.

DIP — Dependency Inversion Principle

Violação 1: Sis cria sqlite3.connect() diretamente (linhas 8–13)

A camada de negócio instancia diretamente a dependência de infraestrutura (SQLite). Não existe abstração entre serviço e banco: trocar o banco de dados exige reescrever Sis.

```
# legacy.py – linhas 8-9
self.db = sqlite3.connect('loja.db') # concreto hard-coded
self.c = self.db.cursor()
```

Impacto: impossível usar banco em memória nos testes sem monkeypatch agressivo.

Violação 2: Notificações implementadas como print() no fluxo de negócio (linhas 40–47)

Notificações estão cravadas como chamadas print() diretas dentro de add_ped e upd_st. Não há abstração de canal: impossível trocar por email real, SMS externo ou qualquer outro mecanismo sem modificar as regras de negócio.

```
# legacy.py – linhas 40-47
if t == 'normal':
    print(f'Email enviado para {n}: Pedido recebido!')
elif t == 'vip':
    print(f'Email enviado para {n}: Pedido recebido!')
    print(f'SMS enviado para {n}: Pedido VIP recebido!')
elif t == 'corporativo':
    print(f'Email enviado para {n}: Pedido recebido!')
    print(f'Notificacao enviada ao gerente de conta de {n}')
```

2 Soluções Implementadas

SRP — Extração em serviços especializados

A classe Sis foi desmembrada em seis serviços com responsabilidade única, cada um recebendo suas dependências via construtor (DIP).

```
# Antes (legacy.py) - tudo em Sis
class Sis:
    def add_ped(self, n, its, t): ... # calcula + persiste + notifica
    def proc_pag(self, id, m, vl): ... # valida + processa + aprova
    def gerar_rel(self, tipo): ... # relatórios

# Depois - cada serviço tem uma responsabilidade
class OrderService:      # ciclo de vida do pedido
class PaymentService:    # processamento de pagamento
class NotificationService: # publicação de eventos
class InventoryService:  # validação de estoque
class ReportService:     # geração de relatórios
class SqliteOrderRepository: # persistência
```

Princípio aplicado: SRP. **Padrão GoF:** Repository (SqliteOrderRepository), Facade (Sis em src/facades/legacy.py).

OCP — Strategy para métodos de pagamento

A cadeia if/elif foi substituída pelo padrão Strategy. Cada método de pagamento é uma classe que implementa IPaymentStrategy. PaymentService delega para o strategy correspondente sem saber qual é.

```
# Antes
if m == 'cartao':
    print('Processando pagamento com cartao...')
    self.upd_st(id, 'aprovado')
    return True

# Depois - src/interfaces/payment_strategy.py
class IPaymentStrategy(ABC):
    def required_amount(self, order_total: float) -> float: ...
    def process(self) -> bool: ...
    @property
    def auto_approves(self) -> bool: ...

# src/strategies/payment_strategy.py
class CartaoStrategy(IPaymentStrategy):
    def process(self) -> bool:
        print('Processando pagamento com cartao...')
        print('Cartao validado!')
        return True

# src/services/payment_service.py
```



```
strategy = self._strategies.get(method) # sem if/elif
success = strategy.process()
```

Princípio aplicado: OCP, DIP. **Padrão GoF:** Strategy.

OCP — Strategy para descontos de itens e clientes

O cálculo de subtotal por item foi encapsulado em `ItemPriceCalculator` e os descontos por tipo de cliente em `IClientDiscountStrategy`.

```
# Antes
if i['tipo'] == 'desc10': tot += i['p'] * i['q'] * 0.9
elif i['tipo'] == 'desc20': tot += i['p'] * i['q'] * 0.8

# Depois - src/strategies/discount_strategy.py
class DefaultItemPriceCalculator(IItemPriceCalculator):
    def calculate(self, item: dict[str, Any]) -> float:
        base: float = float(item['p']) * float(item['q'])
        return base * _ITEM_FACTORS.get(str(item['tipo']), 0.0)

# src/services/order_service.py
subtotal = sum(self._item_calculator.calculate(i) for i in items)
```

Princípio aplicado: OCP, DIP. **Padrão GoF:** Strategy, Decorator (`VolumeDiscountCalculator`).

LSP — Eliminação da herança problemática de `PedEspecial`

`PedEspecial` foi refatorada por composição: não herda mais de `Sis`. Usa `SpecialOrderFactory` que implementa `IOrderFactory`, mantendo o comportamento esperado sem quebrar contratos. A violação de Liskov foi eliminada por design.

```
# Antes
class PedEspecial(Sis):
    def upd_st(self, id, s):
        ...

# Depois - src/facades/legacy.py
class PedEspecial:
    def __init__(self) -> None:
        self._repository = SqliteOrderRepository('loja.db')
        self._factory = SpecialOrderFactory(self._repository)

    def add_ped(self, n, its, t) -> int:
        return self._factory.create(n, its, t)
```

Princípio aplicado: LSP, SRP. **Padrão GoF:** Factory Method (`SpecialOrderFactory`), Facade.

ISP — Criação de 8 interfaces focadas

Foram criadas oito interfaces ABCs em `src/interfaces/`, cada uma cobrindo exatamente o contrato necessário para seu colaborador:

- `IOrderRepository`: operações de persistência de pedidos
- `IPaymentStrategy`: processamento de um método de pagamento
- `IItemPriceCalculator`: cálculo de subtotal de um item
- `IClientDiscountStrategy`: desconto aplicado ao total por tipo de cliente
- `IOrderFactory`: criação de pedidos
- `IOrderEventObserver`: reação a eventos do ciclo de vida do pedido
- `INotifier`: envio de mensagens por canal
- `IInventoryProvider`: consulta de disponibilidade de estoque

Princípio aplicado: ISP, DIP.

DIP — Injeção de dependência explícita em todos os serviços

Todos os serviços recebem suas dependências via construtor, dependendo de abstrações e não de concretos.

```
# Antes - Sis cria suas próprias dependências
self.db = sqlite3.connect('loja.db')    # concreto hard-coded

# Depois - OrderService recebe abstrações no construtor
class OrderService:
    def __init__(
        self,
        repository: IOrderRepository,
        notification_service: NotificationService,
        item_calculator: Optional[IItemPriceCalculator] = None,
        client_discounts: Optional[dict[str, IClientDiscountStrategy]] =
None,
    ) -> None: ...
```

O mesmo padrão é aplicado em `PaymentService`, `InventoryService`, `ReportService`, `EmailOrderObserver` e demais colaboradores.

Princípio aplicado: DIP. **Padrão GoF:** Observer (`NotificationService` + `IOrderEventObserver`).

3 Padrões GoF Aplicados

Foram aplicados cinco padrões GoF. Para cada um são apresentados: nome, intenção segundo o livro Design Patterns (GoF), classes e interfaces criadas, justificativa de aplicação e diagrama parcial.

3.1 Strategy

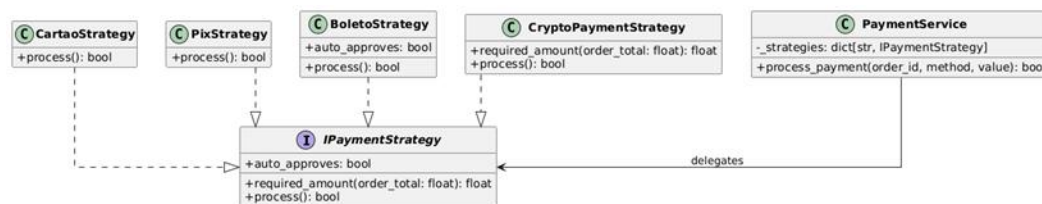
Intenção (GoF): Definir uma família de algoritmos, encapsular cada um e torná-los intercambiáveis. Permite variar o algoritmo independentemente dos clientes que o utilizam.

Classes e interfaces criadas:

- `IPaymentStrategy` (ABC): contrato: `required_amount()`, `process()`, `auto_approves`
- `CartaoStrategy`, `PixStrategy`, `BoletoStrategy`, `CryptoPaymentStrategy` — implementações concretas
- `IItemPriceCalculator` (ABC): contrato: `calculate(item)`
- `DefaultItemPriceCalculator`: cálculo base por tipo de item
- `IClientDiscountStrategy` (ABC): contrato: `apply(total)`
- `VipClientDiscount`, `CorporativoClientDiscount`, `NoClientDiscount`: descontos por tipo de cliente

Justificativa: O legado usava cadeias if/elif hard-coded para pagamentos e descontos. Qualquer novo método (ex: criptomoeda) exigia modificar `Sis` diretamente, violando OCP. Com Strategy, `PaymentService` e `OrderService` recebem a implementação via construtor e nunca precisam ser alterados para suportar novos algoritmos.

Diagrama parcial:



3.2 Repository

Intenção (GoF / Fowler PEAA): Mediar entre o domínio e a camada de mapeamento de dados usando uma interface semelhante a uma coleção para acessar objetos do domínio. Isola a lógica de negócio dos detalhes de persistência.

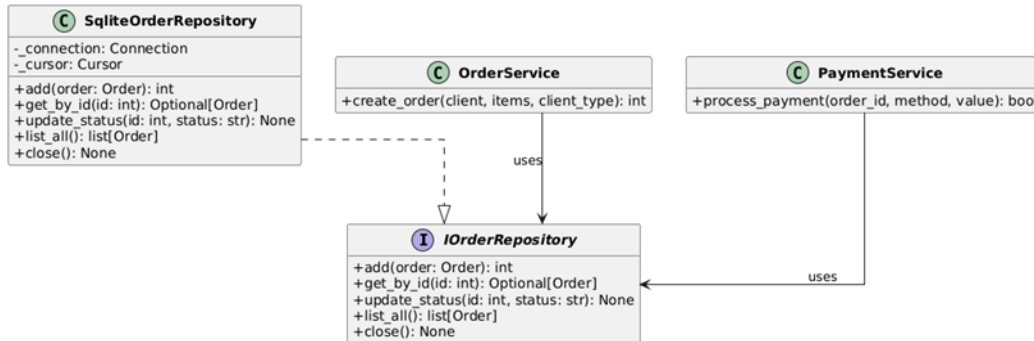
Classes e interfaces criadas:

- `IOrderRepository` (ABC): contrato: `add()`, `get_by_id()`, `update_status()`, `list_all()`, `close()`
- `SqliteOrderRepository`: implementação concreta com SQLite

Justificativa: No legado, `Sis` criava a conexão SQLite no construtor e executava SQL diretamente em todos os métodos. Qualquer serviço dependia de SQLite real para executar.

SqliteOrderRepository encapsula todo o acesso a dados; os serviços dependem apenas de IOrderRepository, permitindo testes com banco em memória sem monkeypatch.

Diagrama parcial:



3.3 Observer (Pub-Sub)

Intenção (GoF): Definir uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Classes e interfaces criadas:

- IOrderEventObserver (ABC): contrato: `on_order_received()`, `on_order_approved()`, `on_order_sent()`, `on_order_delivered()`
- NotificationService publisher: mantém lista de observers, publica eventos
- EmailOrderObserver, SmsOrderObserver, AccountManagerOrderObserver: observers concretos por canal
- PointsOrderObserver: observer para pontos de fidelidade
- WhatsAppOrderObserver: observer adicional (extensão OCP)

Justificativa: No legado, notificações eram `print()` hard-coded dentro de `add_ped` e `upd_st`, duplicados para cada tipo de cliente e cada transição de estado. Com Observer, **OrderService** e **PaymentService** apenas publicam eventos no **NotificationService**; qualquer novo canal (WhatsApp, push) é adicionado registrando um novo observer sem tocar nas regras de negócio.

Diagrama parcial:



3.4 Factory Method

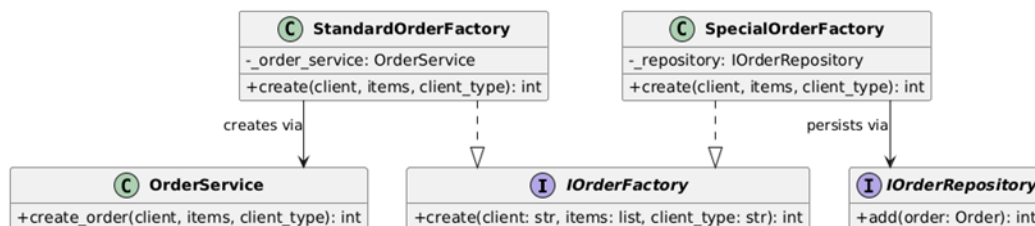
Intenção (GoF): Definir uma interface para criar um objeto, mas deixar as subclasses decidirem qual classe instanciar. Permite que uma classe adie a instanciação para subclasses.

Classes e interfaces criadas:

- IOrderFactory (ABC): contrato: create(client, items, client_type): int
- StandardOrderFactory: delega criação ao OrderService (pedidos normais/vip/corporativo)
- SpecialOrderFactory: cria pedidos com taxa especial de 15% via cálculo próprio

Justificativa: PedEspecial herdava Sis e sobrescrevia add_ped com lógica incompatível, violando LSP. Com Factory Method, a criação de cada tipo de pedido é encapsulada em sua própria factory. A facade PedEspecial usa SpecialOrderFactory por composição sem herdar nada de Sis, eliminando a violação de Liskov.

Diagrama parcial:



3.5 Decorator

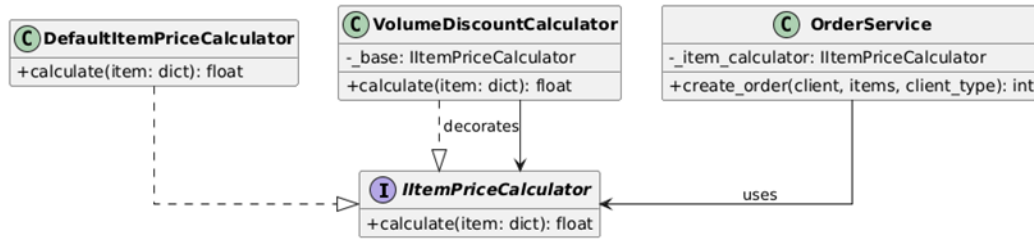
Intenção (GoF): Agregar responsabilidades adicionais a um objeto dinamicamente. Decorators oferecem uma alternativa flexível à herança para estender funcionalidades.

Classes e interfaces criadas:

- ItemPriceCalculator (ABC): componente base do Decorator
- DefaultItemPriceCalculator: componente concreto base
- VolumeDiscountCalculator: decorator: envolve qualquer ItemPriceCalculator e aplica 15% off para $q \geq 3$

Justificativa: O desconto progressivo por volume (Extensão 3) precisava ser adicionado sem modificar DefaultItemPriceCalculator. VolumeDiscountCalculator envolve o calculator base e adiciona a regra de volume apenas quando injetado. Injetar o decorator no OrderService ativa a regra, não injetar mantém o comportamento original, sem nenhuma modificação de código existente.

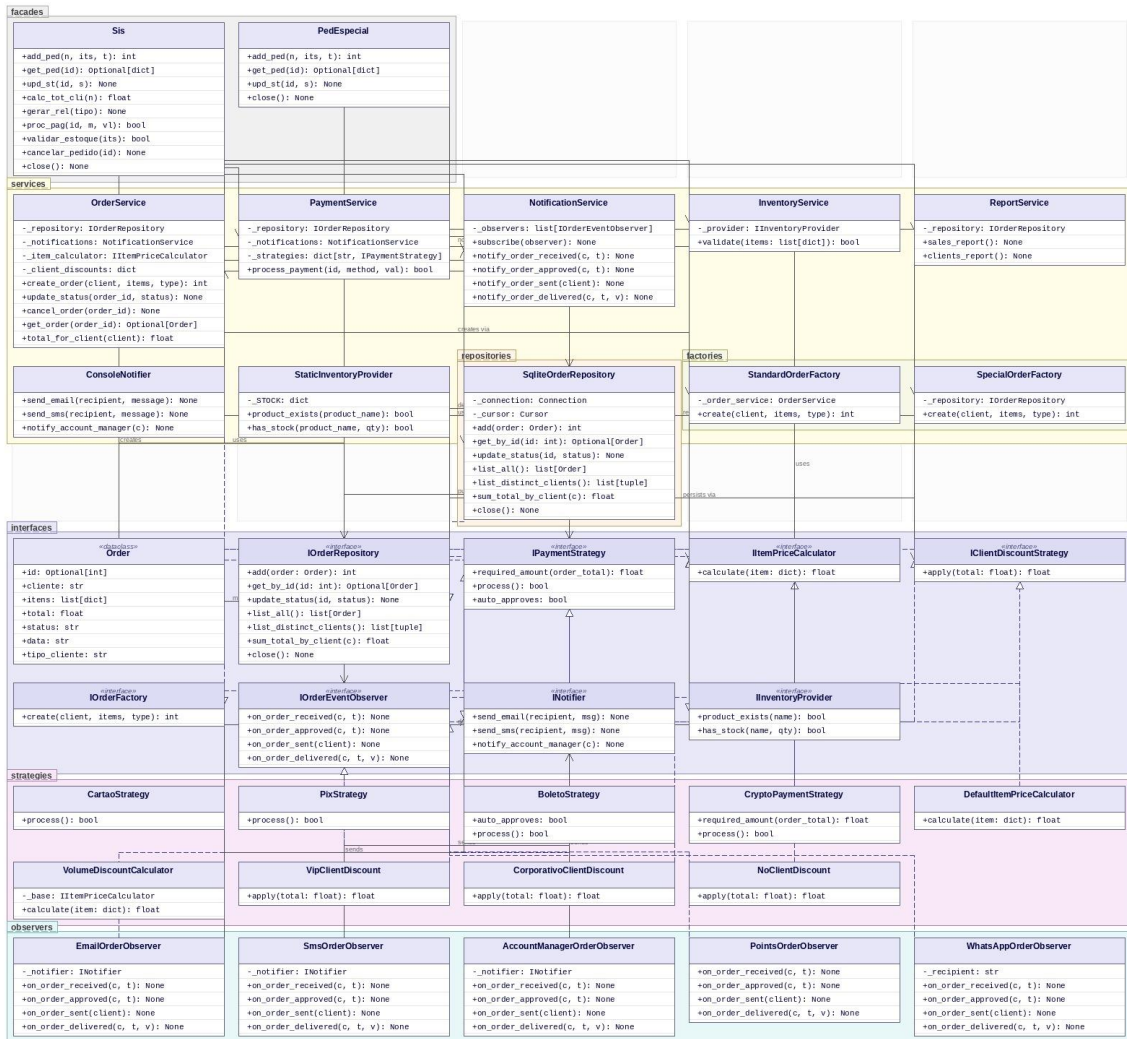
Diagrama parcial:



4 Diagrama UML de Classes

O diagrama completo está em docs/diagrama.puml (PlantUML). A arquitetura é organizada em seis camadas com dependências respeitando DIP (setas sempre apontam de concretos para abstrações):

- **models:** Order (dataclass — dado puro, sem lógica)
- **interfaces:** 8 ABCs: `IOrderRepository`, `IPaymentStrategy`, `ItemPriceCalculator`, `IClientDiscountStrategy`, `IOrderFactory`, `IOrderEventObserver`, `INotifier`, `IInventoryProvider`
- **repositories:** `SqliteOrderRepository` implementa `IOrderRepository`
- **services:** `OrderService`, `PaymentService`, `NotificationService`, `InventoryService`, `ReportService` — dependem apenas de interfaces
- **strategies:** `CartaoStrategy`, `PixStrategy`, `BoletoStrategy`, `CryptoPaymentStrategy`, `DefaultItemPriceCalculator`, `VolumeDiscountCalculator`, `VipClientDiscount`, `CorporativoClientDiscount`
- **observers:** `EmailOrderObserver`, `SmsOrderObserver`, `AccountManagerOrderObserver`, `PointsOrderObserver`, `WhatsAppOrderObserver`
- **factories:** `StandardOrderFactory`, `SpecialOrderFactory` implementam `IOrderFactory`
- **facades:** `Sis` e `PedEspecial` mantêm API legada via composição, camada de compatibilidade



5 Melhorias de Clean Code

Nomenclatura

O código legado usava identificadores de uma única letra ou abreviações opacas. Todos foram renomeados para nomes que revelam intenção:

Antes	Depois	Contexto
Sis	OrderService / PaymentService / etc.	classe monolítica → serviços
n	client	parâmetro de nome do cliente
its	items	lista de itens do pedido
t	client_type	tipo de cliente
m	method	método de pagamento
vl	value	valor do pagamento
s	status / new_status	novo status do pedido
r	order / row	resultado de query SQL
tot	total / subtotal	valor calculado
c	_cursor	cursor do banco de dados

Extração de métodos

- **OrderService._calculate_total()**: separa cálculo de total da persistência
- **OrderService._notify_status_change()**: isola despacho de notificações por status
- **SpecialOrderFactory._calculate_special_total()**: encapsula taxa acumulativa do pedido especial
- **SqliteOrderRepository._row_to_order()**: converte tupla SQL para objeto Order

Eliminação de duplicação

A lógica de notificação estava duplicada em add_ped (linhas 39–47) e em upd_st (linhas 64–80) do legado. Após a refatoração, toda notificação passa pelo NotificationService que publica eventos para os observers registrados, eliminando completamente a duplicação.

Abstrações criadas

Oito interfaces foram criadas em src/interfaces/, tornando cada colaboração externa explícita e substituível. A injeção de dependência via construtor em todos os serviços torna as dependências visíveis e testáveis.

6 Extensões Implementadas

As três extensões obrigatórias foram implementadas sem modificar nenhuma classe existente, adicionando apenas novos arquivos. Isso constitui a prova empírica de OCP da arquitetura refatorada.

Extensão 1 — Pagamento em criptomoeda (taxa de 2%)

Arquivo adicionado: src/strategies/crypto_payment_strategy.py

Arquivo de testes: tests/unit/test_crypto_payment.py (4 testes)

Nenhuma classe existente foi modificada.

```
# src/strategies/crypto_payment_strategy.py
class CryptoPaymentStrategy(IPaymentStrategy):
    def required_amount(self, order_total: float) -> float:
        return order_total * (1 + 0.02)    # 2% de taxa

    def process(self) -> bool:
        print('Processando pagamento em criptomoeda...')
        print('Transacao de cripto confirmada!')
        return True
```

Para ativar o método 'cripto', basta registrá-lo no dicionário de strategies ao instanciar PaymentService, sem tocar em nenhuma classe existente. A arquitetura tornou esta extensão trivial porque IPaymentStrategy define o contrato e PaymentService nunca conhece a implementação concreta.

Extensão 2 — Notificação WhatsApp para todos os tipos de cliente

Arquivo adicionado: src/observers/whatsapp_observer.py

Arquivo de testes: tests/unit/test_whatsapp_notification.py (6 testes)

Nenhuma classe existente foi modificada.

```
# src/observers/whatsapp_observer.py
class WhatsAppOrderObserver(IOrderEventObserver):
    def __init__(self, recipient: str) -> None:
        self._recipient = recipient

    def on_order_received(self, client: str, client_type: str) -> None:
        print(f'WhatsApp enviado para {client}: Pedido recebido!')

    # on_order_approved, on_order_sent, on_order_delivered - todos
    implementados
```

Para ativar o WhatsApp, basta chamar notifications.subscribe(WhatsAppOrderObserver(...)). NotificationService nunca precisou ser tocado. O padrão Observer permite adicionar novos canais registrando novos observers.

Extensão 3 — Desconto progressivo por volume (3+ unidades = 15% off)

Arquivo adicionado: src/strategies/volume_discount_strategy.py

Arquivo de testes: tests/unit/test_volume_discount.py (5 testes)

Nenhuma classe existente foi modificada.

```
# src/strategies/volume_discount_strategy.py
class VolumeDiscountCalculator(IItemPriceCalculator):
    def __init__(self, base: IItemPriceCalculator) -> None:
        self._base = base

    def calculate(self, item: dict[str, Any]) -> float:
        subtotal = self._base.calculate(item)
        if item['q'] >= 3:
            return subtotal * 0.85    # 15% off
        return subtotal
```

VolumeDiscountCalculator usa o padrão Decorator. envolve qualquer IItemPriceCalculator existente sem modificá-lo. Injetar no OrderService ativa a regra; não injetar mantém o comportamento original, provado pelo teste test_desconto_volume_golden_master_nao_afetado.

7 Saída dos Testes

Os testes foram executados com pytest 9.0.3 sobre Python 3.13.1. A suíte é composta por 48 Golden Master Tests que capturam o comportamento do sistema legado e 15 testes unitários das três extensões obrigatórias.

Resultado da execução

```
$ py -3.13 -m pytest --tb=no -q

===== test session starts
=====
platform win32 -- Python 3.13.1, pytest-9.0.3, pluggy-1.6.0
testpaths: tests
collected 63 items

tests/golden_master/test_legacy_behavior.py ..... [ 76%]
tests/unit/test_crypto_payment.py .... [ 82%]
tests/unit/test_volume_discount.py ..... [ 90%]
tests/unit/test_whatsapp_notification.py ..... [100%]

===== 63 passed in 0.78s
=====
```

Cobertura de código (coverage.py)

Cobertura total: 95% (519 statements, 24 não cobertos). Os arquivos não cobertos a 100% são justificados: src/main.py tem 0% pois é apenas ponto de entrada executado via módulo. src/facades/legacy.py tem 76% pois os caminhos de erro da API legada não são exercitados pelos Golden Master Tests.

Name	Stmts	Miss	Cover
-----	-----	-----	-----
src/facades/legacy.py	90	22	76%
src/factories/order_factory.py	29	0	100%
src/observers/notification_observer.py	54	0	100%
src/observers/whatsapp_observer.py	12	0	100%
src/repositories/sqlite_order_repository.py	41	0	100%
src/services/inventory_service.py	21	0	100%
src/services/notification_service.py	26	0	100%
src/services/order_service.py	43	1	98%
src/services/payment_service.py	29	0	100%
src/services/report_service.py	23	0	100%
src/strategies/crypto_payment_strategy.py	9	0	100%
src/strategies/discount_strategy.py	17	0	100%
src/strategies/payment_strategy.py	19	0	100%
src/strategies/volume_discount_strategy.py	12	0	100%
-----	-----	-----	-----
TOTAL	519	24	95%

8 Relatório de Métricas

Lint — Ruff

O linter ruff foi executado com as regras padrão PEP 8 sobre todo o pacote src/. Nenhuma violação foi encontrada.

```
$ py -3.13 -m ruff check src/  
All checks passed!
```

Tipagem estática — mypy --strict

O mypy foi executado no modo strict, que exige anotações de tipo em todos os métodos públicos e privados e proíbe retornos Any implícitos. Nenhum erro foi encontrado nos 31 arquivos fonte.

```
$ py -3.13 -m mypy --strict --explicit-package-bases src/  
Success: no issues found in 31 source files
```

Complexidade ciclomática — Radon CC

A complexidade ciclomática foi medida com radon cc sobre todo o pacote src/. Foram analisados 152 blocos (classes, funções e métodos). A média geral ficou em A (1.55), bem abaixo do limite B aceitável.

Único método acima de A: PaymentService.process_payment obteve B (CC=6) por precisar tratar os casos de pedido inexistente, valor insuficiente, aprovação automática e método inválido em um único fluxo. Ainda assim B é considerado complexidade baixa.

```
$ py -3.13 -m radon cc src/ -a -s  
  
src/services/payment_service.py  
    M 33:4 PaymentService.process_payment - B (6)    <- unico acima de A  
  
152 blocks analyzed.  
Average complexity: A (1.55)
```

Resumo das métricas

Métrica	Ferramenta	Critério	Resultado
Cobertura de testes	coverage.py	maior ou igual a 85%	95% (63/63 testes)
Lint / PEP 8	ruff	0 erros	0 erros
Tipagem estática	mypy --strict	0 erros	0 erros em 31 arquivos
Complexidade média	radon cc	media A ou B	A (1.55)
Testes passando	pytest	100%	63/63

9 Reflexão Metacognitiva

O princípio que mais gerou dúvida

De todos os princípios estudados, o LSP foi o que mais gerou discussão no grupo. A definição formal, subtipos devem ser substituíveis pelo tipo base sem alterar o comportamento do programa, parece simples, mas na prática é bem mais sutil.

Quando olhamos para PedEspecial pela primeira vez, a impressão era de que estava tudo certo. A classe compilava, os testes passavam, e o sistema funcionava do jeito esperado. Só depois de analisar com mais cuidado percebemos que o `upd_st` da subclasse simplesmente ignorava as notificações que Sis prometia disparar, e que um cliente VIP usando PedEspecial perdia o desconto de 5% que teria direito. Isso não é 'comportamento especializado', é quebra de contrato.

O que ficou de aprendizado é que LSP não é só sobre assinatura de método. Uma subclasse pode ter os mesmos parâmetros e retorno do pai e ainda assim violar o princípio se as pós-condições forem mais fracas. Ainda temos dúvida sobre casos limítrofes. Por exemplo, se uma subclasse lança uma exceção aonde o pai simplesmente retornaria `None`, isso viola LSP? O critério comportamental ainda não está completamente consolidado para nós.

Como usamos IA neste trabalho

Identificação inicial das violações: pedimos para a IA analisar o `legacy.py` e apontar as violações SOLID. Ela foi útil para ter um ponto de partida rápido, mas nem tudo que ela identificou estava correto. Por exemplo, a IA apontou o método `cancelar_pedido` como violação de SRP por não validar regras de negócio, mas no contexto do sistema legado isso era um comportamento intencional documentado com `TODO`, não era a violação mais relevante a destacar. Também classificou a ausência de `type hints` como violação de ISP, o que não faz sentido. ISP é sobre segregação de interfaces, não sobre tipagem. Essas discordâncias nos obrigaram a voltar ao material da disciplina e entender o princípio de forma mais precisa antes de escrever a análise.

Documentação: após alinharmos as violações reais, usamos a IA para gerar uma versão inicial deste documento. Ela foi especialmente útil para escrever os trechos de código com as referências corretas de arquivo e número de linha. O texto foi revisado e reescrito pelo grupo em vários pontos onde a IA soava genérica demais ou não refletia as decisões que tomamos de fato no projeto.

Auxílio durante a programação: durante a implementação, usamos a IA para tirar dúvidas pontuais de forma mais rápida do que seria buscar na documentação ou em fóruns. Questões como a diferença entre `@abstractmethod` e `@property` combinados, e como configurar o `mypy --strict` para projetos com múltiplos pacotes foram resolvidas em minutos com a IA. Isso acelerou bastante o ritmo de desenvolvimento, especialmente nas partes mais técnicas da refatoração.

Bibliografia

Martin, R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008. Capítulos 2, 3, 6, 10.

Feathers, M. Working Effectively with Legacy Code. Prentice Hall, 2004. Capítulos 13 (GoldenMaster) e 22.

Gamma, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994. Strategy, Observer, Factory Method.

Fowler, M. Refactoring: Improving the Design of Existing Code. 2a ed. Addison-Wesley, 2018.

5. North, D. CUPID for joyful coding. Disponível em <https://dannorth.net/cupid-for-joyful-coding/>.